



# Bookstore E-Commerce

## Microservices Architecture

*Full Documentation — Spring Boot & Spring Cloud*

| Component            | Technology                               |
|----------------------|------------------------------------------|
| Language / Framework | Java 17+ / Spring Boot 3.x               |
| Service Discovery    | Netflix Eureka (Spring Cloud)            |
| API Gateway          | Spring Cloud Gateway                     |
| Config Management    | Spring Cloud Config Server               |
| Auth / Security      | Spring Security + JWT                    |
| Messaging            | Apache Kafka / RabbitMQ                  |
| Databases            | PostgreSQL (per service) + Redis (cache) |
| Documentation        | SpringDoc OpenAPI 3 / Swagger UI         |
| Containerization     | Docker + Docker Compose                  |
| Orchestration        | Kubernetes (optional)                    |

## 1. Microservices Overview

The monolithic Bookstore Back-end API is decomposed into 9 independent microservices. Each service owns its own database, exposes RESTful APIs, and communicates over HTTP (synchronous) or Kafka (asynchronous). All external traffic is routed through a single API Gateway.

| # | Microservice             | Port | Database        | Responsibility                               |
|---|--------------------------|------|-----------------|----------------------------------------------|
| 1 | User Service             | 8081 | user_db         | Registration, login, JWT issuance, profile   |
| 2 | Admin Service            | 8082 | admin_db        | Admin management, role assignment, dashboard |
| 3 | Product Service          | 8083 | product_db      | CRUD for books, categories, inventory        |
| 4 | Cart Service             | 8084 | cart_db (Redis) | Add/remove items, cart total calculation     |
| 5 | WishList Service         | 8085 | wishlist_db     | Add/remove wish-list items per user          |
| 6 | Customer Details Service | 8086 | customer_db     | Addresses, phone, preferences                |
| 7 | Order Service            | 8087 | order_db        | Place orders, track status, order history    |
| 8 | Feedback Service         | 8088 | feedback_db     | Submit & retrieve product reviews/ratings    |
| 9 | Notification Service     | 8089 | —               | Email/SMS notifications via Kafka events     |

## 2. Infrastructure Services

### 2.1 API Gateway (Port 8080)

All client requests enter via Spring Cloud Gateway. It handles routing, JWT validation, rate limiting, and CORS.

```
# application.yml - API Gateway
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://USER-SERVICE
          predicates:
            - Path=/api/users/**
          filters:
            - AuthenticationFilter
        - id: product-service
          uri: lb://PRODUCT-SERVICE
          predicates:
            - Path=/api/products/**
        - id: order-service
```

```
uri: lb://ORDER-SERVICE
predicates:
  - Path=/api/orders/**
```

## 2.2 Eureka Service Discovery (Port 8761)

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

## 2.3 Config Server (Port 8888)

Centralises all application.yml files. Each microservice fetches its config on startup from the Config Server (backed by a Git repo or classpath).

# 3. Microservice Details

---

## 3.1 User Service

### Responsibility

Handles user registration, login, JWT generation, and profile management. Acts as the authentication authority for the entire platform.

### Maven Dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.3</version>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

## REST API Endpoints

| Method | Endpoint                   | Auth Required | Description                        |
|--------|----------------------------|---------------|------------------------------------|
| POST   | /api/users/register        | No            | Register a new user account        |
| POST   | /api/users/login           | No            | Authenticate and receive JWT token |
| GET    | /api/users/profile         | JWT           | Fetch authenticated user profile   |
| PUT    | /api/users/profile         | JWT           | Update user profile information    |
| PUT    | /api/users/change-password | JWT           | Change account password            |
| DELETE | /api/users/{id}            | ADMIN         | Delete a user account (admin only) |

## Entity: User.java

```
@Entity
@Table(name = "users")
@Data @NoArgsConstructor @AllArgsConstructor
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String email;

    @JsonIgnore
    private String password;

    @Enumerated(EnumType.STRING)
    private Role role; // USER | ADMIN

    private LocalDateTime createdAt;
}
```

## JwtUtil.java

```
@Component
public class JwtUtil {
    @Value("${jwt.secret}") private String secret;
    @Value("${jwt.expiry}") private long expiry;

    public String generateToken(UserDetails user) {
        return Jwts.builder()
            .subject(user.getUsername())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expiry))
            .signWith(Keys.hmacShaKeyFor(secret.getBytes()))
            .compact();
    }
}
```

```

    }

    public String extractUsername(String token) {
        return Jwts.parser().verifyWith(...).build()
            .parseSignedClaims(token).getPayload().getSubject();
    }
}

```

## 3.2 Admin Service

### Responsibility

Manages admin accounts, role assignments, and provides admin-only product management endpoints. Calls Product Service via Feign Client.

### REST API Endpoints

| Method | Endpoint                      | Auth Required | Description                    |
|--------|-------------------------------|---------------|--------------------------------|
| POST   | /api/admin/register           | SUPER_ADMIN   | Create new admin account       |
| GET    | /api/admin/all-users          | ADMIN         | List all registered users      |
| PUT    | /api/admin/products/{id}      | ADMIN         | Update any product (via Feign) |
| DELETE | /api/admin/products/{id}      | ADMIN         | Delete a product (via Feign)   |
| GET    | /api/admin/orders             | ADMIN         | View all orders platform-wide  |
| PUT    | /api/admin/orders/{id}/status | ADMIN         | Update order status            |

### Feign Client — ProductServiceClient.java

```

@FeignClient(name = "PRODUCT-SERVICE", path = "/api/products")
public interface ProductServiceClient {

    @PutMapping("/{id}")
    ProductResponse updateProduct(@PathVariable Long id,
                                  @RequestBody ProductRequest req);

    @DeleteMapping("/{id}")
    void deleteProduct(@PathVariable Long id);
}

```

## 3.3 Product Service

### Responsibility

Full CRUD for books/products including category management, inventory tracking, and image URL storage. This is the most-read service on the platform.

REST API Endpoints

| Method | Endpoint                    | Auth Required | Description                     |
|--------|-----------------------------|---------------|---------------------------------|
| GET    | /api/products               | No            | List all products (paginated)   |
| GET    | /api/products/{id}          | No            | Get product by ID               |
| GET    | /api/products/search?q=     | No            | Search products by title/author |
| GET    | /api/products/category/{id} | No            | Filter products by category     |
| POST   | /api/products               | ADMIN         | Create a new product            |
| PUT    | /api/products/{id}          | ADMIN         | Update product details          |
| DELETE | /api/products/{id}          | ADMIN         | Delete a product                |
| GET    | /api/categories             | No            | List all categories             |
| POST   | /api/categories             | ADMIN         | Create a new category           |

Entity: Product.java

```
@Entity @Data @NoArgsConstructor @AllArgsConstructor
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String author;
    private String isbn;
    private BigDecimal price;
    private Integer stockQuantity;
    private String imageUrl;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category;
}
```

3.4 Cart Service

Responsibility

Manages shopping cart sessions stored in Redis for ultra-fast read/write. Validates product existence via Feign Client to Product Service.

## REST API Endpoints

| Method | Endpoint                     | Auth Required | Description                |
|--------|------------------------------|---------------|----------------------------|
| GET    | /api/cart                    | JWT           | Get current user's cart    |
| POST   | /api/cart/add                | JWT           | Add item to cart           |
| PUT    | /api/cart/update             | JWT           | Update item quantity       |
| DELETE | /api/cart/remove/{productId} | JWT           | Remove item from cart      |
| DELETE | /api/cart/clear              | JWT           | Clear entire cart          |
| GET    | /api/cart/total              | JWT           | Calculate cart total price |

## Redis Cart Model

```
@RedisHash("cart")
public class Cart {
    @Id
    private String userId; // key: cart:{userId}

    private List<CartItem> items = new ArrayList<>();
    private BigDecimal totalAmount;
}

public class CartItem {
    private Long productId;
    private String productTitle;
    private int quantity;
    private BigDecimal unitPrice;
}
```

## 3.5 WishList Service

### Responsibility

Persists user wish lists in PostgreSQL. Each user can maintain one wish list with multiple products.

## REST API Endpoints

| Method | Endpoint                         | Auth Required | Description                        |
|--------|----------------------------------|---------------|------------------------------------|
| GET    | /api/wishlist                    | JWT           | Get authenticated user's wish list |
| POST   | /api/wishlist/add/{productId}    | JWT           | Add product to wish list           |
| DELETE | /api/wishlist/remove/{productId} | JWT           | Remove product from wish list      |

| Method | Endpoint            | Auth Required | Description            |
|--------|---------------------|---------------|------------------------|
| DELETE | /api/wishlist/clear | JWT           | Clear entire wish list |

### 3.6 Customer Details Service

#### Responsibility

Stores and manages delivery addresses, phone numbers, and personal preferences for registered users.

#### REST API Endpoints

| Method | Endpoint                              | Auth Required | Description                  |
|--------|---------------------------------------|---------------|------------------------------|
| GET    | /api/customers/details                | JWT           | Get customer profile details |
| POST   | /api/customers/details                | JWT           | Create customer profile      |
| PUT    | /api/customers/details                | JWT           | Update customer profile      |
| POST   | /api/customers/addresses              | JWT           | Add a new delivery address   |
| GET    | /api/customers/addresses              | JWT           | List all saved addresses     |
| DELETE | /api/customers/addresses/{id}         | JWT           | Delete a saved address       |
| PUT    | /api/customers/addresses/{id}/default | JWT           | Set default address          |

### 3.7 Order Service

#### Responsibility

Handles the full order lifecycle — placement, payment verification, status transitions, and history. Publishes Kafka events on order placement and status change.

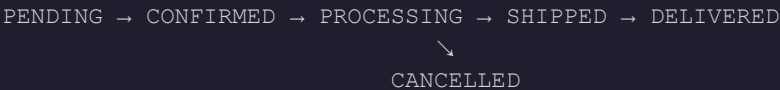
#### REST API Endpoints

| Method | Endpoint                | Auth Required | Description                 |
|--------|-------------------------|---------------|-----------------------------|
| POST   | /api/orders             | JWT           | Place a new order from cart |
| GET    | /api/orders             | JWT           | List current user's orders  |
| GET    | /api/orders/{id}        | JWT           | Get order details by ID     |
| PUT    | /api/orders/{id}/cancel | JWT           | Cancel a pending order      |



| Method | Endpoint                | Auth Required | Description             |
|--------|-------------------------|---------------|-------------------------|
| GET    | /api/orders/all         | ADMIN         | List all orders (admin) |
| PUT    | /api/orders/{id}/status | ADMIN         | Update order status     |

### Order Status Flow



### Kafka Event Publisher

```
@Service
public class OrderEventPublisher {
    @Autowired
    private KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public void publishOrderPlaced(Order order) {
        OrderEvent event = new OrderEvent(order.getId(), order.getUserId(),
                                           "ORDER_PLACED", LocalDateTime.now());
        kafkaTemplate.send("order-events", event);
    }
}
```

## 3.8 Feedback Service

### Responsibility

Allows verified buyers to submit product reviews and star ratings. Calculates aggregate ratings per product.

### REST API Endpoints

| Method | Endpoint                          | Auth Required | Description                    |
|--------|-----------------------------------|---------------|--------------------------------|
| POST   | /api/feedback                     | JWT           | Submit a product review        |
| GET    | /api/feedback/product/{id}        | No            | Get all reviews for a product  |
| GET    | /api/feedback/product/{id}/rating | No            | Get average rating for product |
| PUT    | /api/feedback/{id}                | JWT           | Edit own review                |
| DELETE | /api/feedback/{id}                | JWT/ADMIN     | Delete a review                |

## 3.9 Notification Service

### Responsibility

Listens to Kafka topics and sends transactional emails/SMS. Decoupled from all other services — purely event-driven.

### Kafka Consumer

```
@Service
public class NotificationConsumer {

    @KafkaListener(topics = "order-events", groupId = "notification-group")
    public void onOrderEvent(OrderEvent event) {
        switch (event.getType()) {
            case "ORDER_PLACED" -> emailService.sendOrderConfirmation(event);
            case "ORDER_SHIPPED" -> emailService.sendShippingUpdate(event);
            case "ORDER_DELIVERED" ->
                emailService.sendDeliveryConfirmation(event);
        }
    }

    @KafkaListener(topics = "user-events", groupId = "notification-group")
    public void onUserEvent(UserEvent event) {
        if ("USER_REGISTERED".equals(event.getType()))
            emailService.sendWelcomeEmail(event);
    }
}
```

## 4. Security Architecture

### 4.1 JWT Authentication Flow

1. Client sends POST `/api/users/login` with credentials.
2. User Service validates credentials, generates a signed JWT, and returns it.
3. Client stores the JWT and sends it in the Authorization: Bearer <token> header on every subsequent request.
4. API Gateway's AuthenticationFilter validates the JWT signature and expiry before forwarding the request.
5. The JWT claims (userId, role) are propagated as request headers to downstream services.

### 4.2 Role-Based Access Control

| Role        | Accessible Services                                | Restrictions                  |
|-------------|----------------------------------------------------|-------------------------------|
| GUEST       | Product (read), Feedback (read)                    | No cart, wishlist, or orders  |
| USER        | All user-facing services                           | Cannot access admin endpoints |
| ADMIN       | Admin Service, all read + write on products/orders | Cannot modify other admins    |
| SUPER_ADMIN | Full access including admin management             | System-level operations only  |

### 4.3 Security Config Skeleton

```
@Configuration @EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s ->
                s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/users/register",
                    "/api/users/login").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

## 5. Inter-Service Communication

### 5.1 Synchronous — Feign Client

Used when a service requires a real-time response from another service (e.g., Cart → Product to validate stock).

```
// Add to pom.xml
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

// Enable in main class
@EnableFeignClients

// Example — Order Service calling Cart Service
@FeignClient(name = "CART-SERVICE")
public interface CartServiceClient {
    @GetMapping("/api/cart")
    CartResponse getCart(@RequestHeader("userId") Long userId);

    @DeleteMapping("/api/cart/clear")
    void clearCart(@RequestHeader("userId") Long userId);
}
```

### 5.2 Asynchronous — Apache Kafka

| Topic            | Producer         | Consumer(s)          | Trigger                            |
|------------------|------------------|----------------------|------------------------------------|
| order-events     | Order Service    | Notification Service | Order placed / status changed      |
| user-events      | User Service     | Notification Service | User registered / password changed |
| inventory-events | Order Service    | Product Service      | Reduce stock on order confirmed    |
| review-events    | Feedback Service | Product Service      | Recalculate product rating         |

## 6. Database Design

### 6.1 Database-per-Service Pattern

Each microservice owns its own PostgreSQL schema. Cross-service joins are handled at the application layer by composing API responses.

| Service          | DB Schema   | Key Tables                                |
|------------------|-------------|-------------------------------------------|
| User Service     | user_db     | users, roles                              |
| Product Service  | product_db  | products, categories, product_images      |
| Cart Service     | Redis       | cart:{userId} (hash with TTL)             |
| WishList Service | wishlist_db | wishlists, wishlist_items                 |
| Customer Details | customer_db | customer_profiles, addresses              |
| Order Service    | order_db    | orders, order_items, order_status_history |
| Feedback Service | feedback_db | reviews, ratings                          |
| Admin Service    | admin_db    | admins, audit_logs                        |

### 6.2 Common application.yml (per service)

```
spring:
  application:
    name: product-service
  datasource:
    url: jdbc:postgresql://localhost:5432/product_db
    username: ${DB_USER}
    password: ${DB_PASS}
  jpa:
    hibernate.ddl-auto: update
    show-sql: false
  cloud:
    config:
      uri: http://config-server:8888

eureka:
  client:
    service-url:
      defaultZone: http://eureka:8761/eureka/

springdoc:
  api-docs.path: /api-docs
  swagger-ui.path: /swagger-ui.html
```

## 7. Exception Handling & Validation

---

### 7.1 Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException
ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(404, ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
handleValidation(MethodArgumentNotValidException ex) {
        String msg = ex.getBindingResult().getFieldErrors().stream()
            .map(e -> e.getField() + ": " + e.getDefaultMessage())
            .collect(Collectors.joining(", "));
        return ResponseEntity.badRequest().body(new ErrorResponse(400, msg));
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse(500, "Internal server
error"));
    }
}
```

### 7.2 Input Validation Example

```
public class UserRegistrationRequest {
    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Invalid email format")
    @NotBlank
    private String email;

    @Size(min = 8, message = "Password must be at least 8 characters")
    @Pattern(regexp = ".*[A-Z].*", message = "Password must contain an uppercase
letter")
    private String password;
}
```

## 8. Swagger / OpenAPI Documentation

### 8.1 SpringDoc Configuration

```
@Configuration
public class OpenApiConfig {

    @Bean
    public OpenAPI bookstoreOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Bookstore Microservice - {Service Name}")
                .description("RESTful API documentation")
                .version("v1.0")
                .contact(new Contact().name("Dev Team")))
            .addSecurityItem(new SecurityRequirement().addList("Bearer"))
            .components(new Components().addSecuritySchemes("Bearer",
                new SecurityScheme()
                    .name("Authorization")
                    .type(SecurityScheme.Type.HTTP)
                    .scheme("bearer")
                    .bearerFormat("JWT")));
    }
}
```

### 8.2 Annotating Controllers

```
@Tag(name = "Products", description = "Product management APIs")
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Operation(summary = "Get product by ID",
        responses = {
            @ApiResponse(responseCode = "200", description = "Product found"),
            @ApiResponse(responseCode = "404", description = "Not found")})
    @GetMapping("/{id}")
    public ResponseEntity<ProductResponse> getById(@PathVariable Long id) { ... }
}
```

#### Swagger UI URLs

Each microservice exposes Swagger UI at `http://localhost:{PORT}/swagger-ui.html`. The API Gateway can aggregate all docs at `http://localhost:8080/swagger-ui.html` using Springdoc gateway aggregation.

## 9. Docker & Deployment

---

### 9.1 Dockerfile Template

```
FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/*.jar app.jar
EXPOSE 8083
ENTRYPOINT ["java", "-jar", "app.jar"]
```

### 9.2 Docker Compose (Excerpt)

```
version: "3.9"
services:
  eureka:
    build: ./eureka-server
    ports: ["8761:8761"]

  api-gateway:
    build: ./api-gateway
    ports: ["8080:8080"]
    depends_on: [eureka, config-server]

  product-service:
    build: ./product-service
    ports: ["8083:8083"]
    environment:
      DB_URL: jdbc:postgresql://product-db:5432/product_db
      EUREKA_URL: http://eureka:8761/eureka/
    depends_on: [product-db, eureka]

  product-db:
    image: postgres:16
    environment:
      POSTGRES_DB: product_db
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: secret

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    ports: ["9092:9092"]
```



## 10. Project Structure

```
bookstore-microservices/
├── eureka-server/
├── config-server/
│   └── src/main/resources/configs/
│       ├── user-service.yml
│       ├── product-service.yml
│       └── ...
├── api-gateway/
├── user-service/
│   └── src/main/java/com/bookstore/user/
│       ├── controller/
│       ├── service/
│       ├── repository/
│       ├── entity/
│       ├── dto/
│       ├── security/
│       └── exception/
├── product-service/
├── cart-service/
├── wishlist-service/
├── customer-details-service/
├── order-service/
├── feedback-service/
├── notification-service/
└── docker-compose.yml
```

## 11. Key Learnings & Best Practices

| Principle             | Implementation                                                   |
|-----------------------|------------------------------------------------------------------|
| Single Responsibility | Each microservice owns exactly one bounded context               |
| Database Isolation    | Separate PostgreSQL schema per service — no shared tables        |
| Stateless Auth        | JWT tokens — no server-side session state required               |
| Async Decoupling      | Kafka events for notifications — services never block each other |
| Fail Fast             | Feign circuit breakers (Resilience4j) prevent cascade failures   |
| Externalized Config   | Spring Cloud Config Server — no hardcoded values in JARs         |
| API Documentation     | SpringDoc auto-generates Swagger UI from annotations             |
| Container First       | Every service ships with a Dockerfile for repeatable deploys     |